

Proyecto 1: Aplicación chat

Nombre del alumno:Gómez Ferro Ana Luisa

Materia: Modelado y Programación

Reporte de desarrollo del proyecto 1: Aplicación chat.

1.-INTRODUCCIÓN.

El siguiente reporte tiene como objetivo describir el proceso de creación de una aplicación chat cliente/servidor, en esta varios clientes serán capaces de comunicarse entre sí a través de un servidor, y tendrán la posibilidad de enviar mensajes públicos, privados, creas salas, etc. Para ello, se ocupará el modelo en cascada, el cual consta de 5 fases, análisis, diseño, implementación, pruebas y mantenimiento, ahondando principalmente, en las fases del Análisis y el Diseño.

2.-ANÁLISIS.

El planteamiento del problema a resolver es el siguiente: Se requiere una aplicación chat que se componga de un programa servidor y un programa cliente, debe existir una comunicación continua entre varios clientes (o usuarios) a través del servidor, que funcionará de manera concurrente. En esta aplicación los clientes serán capaces de realizar múltiples cosas, las cuales se encuentran descritas en el protocolo del proyecto.

A continuación se muestra una lista de posibles acciones que se podrán hacer en la aplicación:

- Se podrán conectar múltiples clientes identificándose con un nombre de usuario, y recibirán la lista de clientes conectados si la conexión es exitosa.
- Los clientes podrán cambiar su estado entre 3 opciones (active, away, o busy) y se le notificará al resto de clientes.
- Los clientes podrán solicitar la lista de clientes conectados en cualquier momento.
- Los clientes podrán mandar mensajes privados entre sí, y mensajes públicos para todos los clientes conectados.
- Los clientes podrán crear una sala en el chat e invitar a uno o múltiples usuarios.
- Para unirse a una sala el cliente deberá ser invitado previamente.
- Las salas desaparecerán cuando ya no haya clientes dentro suyo.
- Cuando un cliente se una a una sala, los clientes dentro de ella serán notificados.
- Se podrá solicitar la lista de clientes de una sala si el cliente que la solicita es parte de ella.
- Un cliente podrá abandonar una sala en cualquier momento.
- Los clientes podrán desconectarse del chat en cualquier momento.

Además, para la comunicación entre los clientes y el servidor se debe usar el formato JSON (Para más detalle consultar el protocolo del proyecto).

Este es un problema ya conocido y que ha tenido varias implementaciones a lo largo del tiempo, e incluso con una complejidad mayor, lo que asegura que el problema se puede resolver mediante la creación de algoritmos, es decir, se sabe que el problema es computable.

Para comprender más a fondo el problema se dividirá en problemas más pequeños. Tenemos que tener 2 programas: un servidor que acepte clientes y realice múltiples operaciones, y un cliente que pueda conectarse al servidor y solicitar varias cosas al mismo, además de tener una manera adecuada de mostrar el chat al usuario.

Del lado del servidor: para poder comunicar entre sí a los clientes y atender sus peticiones se tiene que:

- Poder recibir las conexiones de los clientes, esto se hace mediante el uso de sockets según lo descrito en la especificación del proyecto.
- Poder enviar mensajes al cliente con la conexión que mantenemos con él.
- Solicitar el nombre de usuario de cada cliente e impedir la conexión si ese nombre está en uso.
- Guardar el nombre de usuario de cada cliente y la conexión que tenemos con él.
- Mantener al servidor escuchando las peticiones de los clientes y respondiendo a ellas.
- Mantener una comunicación con el cliente por medio del formato JSON, lo que implica identificar qué tipo de acción solicita el cliente a partir de la información de los campos en el JSON.
- Ser capaz de manejar varios clientes a la vez (usar concurrencia).
- Crear salas y guardar los clientes que estén dentro de ella.
- Comunicar a dos o más clientes por medio de las conexiones que guardamos con cada uno.

Del lado del cliente: Para poder comunicarse con el servidor se tiene que:

- Poder establecer una conexión con el servidor (mediante el uso de sockets)
- Ser capaces de enviar mensajes al servidor con el formato JSON.
- Ser capaces de recibir (o escuchar) los mensajes que envíe el servidor.
- Mantener la lista de los clientes de forma sincronizada con el servidor.
- Mostrar de una manera óptima la aplicación al usuario, de forma que sea fácil de manejar.

Dadas las características del problema, podemos abordarlo usando el paradigma orientado a objetos, teniendo como objetos principales al servidor y al cliente.

Con el servidor podemos tener de atributos: su socket de escucha para recibir clientes, la lista de usuarios y sus respectivas conexiones y el conjunto de cuartos con la listas de usuarios de cada uno. Y como métodos podemos tener algunos como `levantar_servidor()`, `recibir_conexiones()`, `crear_cuarto()`, `obtener_usuarios()`, etc.

Con el cliente podemos tener de atributos su conexión con el servidor, su nombre de usuario, y la lista de clientes en el servidor, y podemos tener metodos como solicitar_conexion(), envair_mensaje(), crear_cuarto(), pedir_usuarios(), mostrar_chat(), etc.

Así, pasamos a la siguiente etapa que es el desarrollo, teniendo presente que la creación de un proyecto no es lineal, y que dependiendo de la evolución de las demás etapas podemos volver a replantear el análisis con nueva información que se haya recabado.

2.-DESARROLLO.

Dado que en el análisis se determinó usar orientación a objetos en este apartado describiremos la planificación de los programas a través de clases, métodos, atributos y demás características propias de este paradigma.

Como se dividió el problema en 2 programas, describiremos primero el programa del servidor:

Para la construcción del servidor, usaremos 4 clases para cubrir los subproblemas planteados en el analisis:

Clase No.1, Servidor:

Tiene como atributos:

- Su socket de escucha para recibir clientes (un entero).
- El puerto que se le asigna al correr el programa (un entero).
- Una instancia de la clase ManejadorEstado.

Tiene como métodos:

- **iniciarServidor():** método que se encarga de crear el socket, asignarle un puerto, ponerlo en modo de escucha, entre otras cosas más específicas con ayuda de las funciones para la programación en redes socket(), bind(), listen() y accept().
- **aceptarClientes():** método que se encarga de estar en escucha continua (con un while) para recibir las conexiones de los clientes, una vez que obtenga una, creará un hilo de ejecución para el cliente y mandará a ejecutar una instancia de la clase ManejadorCliente

Clase No.2 ManejadorCliente:

Tiene como atributos:

- El nombre de usuario del cliente (un string).
- El socket mediante el cual el servidor envía y recibe mensajes.
- El estado del cliente.
- la lista de cuartos a los que pertenece el cliente.
- La instancia de la clase ManejadorEstado que tiene la clase servidor y que le pasa como parámetro.
- Una instancia de la clase ConstructorMensajes.

Tiene como métodos:

- **identificarCliente()**: solicita al cliente su nombre de usuario, si este usuario no ha sido utilizado lo guarda junto a su socket, si ya está en uso desconecta al cliente y termina la ejecución completa. Esto lo hace pidiendo la lista de usuarios a la clase ManejadorEstados y solicitándole que agregue a un nuevo usuario a la lista.
- **escucharCliente()**: método que está en continua escucha (con ayuda de un while) y recibe los mensajes del cliente, si recibe alguno manda a llamar a descifrarMensaje().
- **descifrarMensaje()**: método que recibe el mensaje del cliente, el cual está en formato JSON según el protocolo, ocupa el método desarmarMensaje() de la clase ConstructorMensajes para obtener un diccionario con las especificaciones de la petición y/o alguna lista que haya dado el cliente (para la creación de un cuarto por ejemplo).
- **identificadorDeAccion()**: Toma la estructura que retorna el método descifrarMensaje() y en base a los campos que contengan determina la acción a realizar y manda a llamar al método auxiliar respectivo.

Metodos auxiliares: Se encargaran de ejecutar las acciones específicas que pueda pedir el cliente, algunos de ellos son:

- **mensajePublico()**: Se comunica con la clase ManejadorEstados y solicita el socket de todos los clientes, con los que les envía el mensaje a todos.
- **mensajePrivado()**: Se comunica con la clase ManejadorEstados y solicita el socket del cliente pedido si es que existe, y con el envió el mensaje.
- **cambiarEstado()**: si el nuevo estado es distinto al que hay lo cambia en la clase, se comunica con ManejadorEstados y solicita cambiar el estado del usuario en la lista, y manda a llamar a mensajePublico() para notificarle al resto de clientes.
- **crearSala()**: solicita a ManejadorEstados la creación de una nueva sala y se le da el nombre de la sala y la lista de usuarios (esta última de forma opcional).

... (y sigue)